Paraconsistent Consensus: Federated Meta-Learning Over BlockDAG Finality Checkpoints

Belnap Four-Valued Aggregation, LoRA Adapter Composition, and Recursive Learning

Larry Klosowski Lauren Mendenhall
Citrate Inc.
Larry@citrate.ai

The Gradient Papers, No. II
Preprint — not yet peer reviewed

Maturity: [Implemented] core, [Hypothesis] convergence. In February 2026 this paper was entirely theoretical. The core aggregation primitive is now implemented as an on-chain precompile and unit-tested, the checkpoint-synchronized learning state is committed by the node, and the LoRA lifecycle runs against deployed contracts. This revision re-anchors those claims to source, corrects two parameters, adds one important architectural honesty note (where the four-valued logic actually lives), and keeps the convergence result an explicit conjecture, because it has not been proven.

0.1 Abstract

We present Paraconsistent Consensus, a framework unifying BlockDAG consensus and federated meta-learning into a single protocol. The core contribution is an aggregation function grounded in Belnap's four-valued logic (FOUR), which classifies each node's model output into one of four epistemic states, relevant, irrelevant, contradictory, or unknown, relative to a query, and preserves all four through aggregation rather than collapsing them by averaging. The framework operates atop the Citrate Network's GhostDAG consensus with BFT finality checkpoints (Paper I), extending checkpoint commitments to include learning state. Each node is both a consensus validator and a computational unit in a distributed meta-learning architecture. A lightweight routing model, a small attention layer or MLP, is trained at checkpoint intervals to learn input-dependent routing across heterogeneous node models. For adaptation we use LoRA-based adapter generation and composition, and we give an honest assessment of interference bounds: frozen base weights provide structural protection against catastrophic forgetting, while multi-adapter composition introduces interference that requires active mitigation. Unlike the February 2026 draft, the aggregation primitive, the learning checkpoint, and the adapter lifecycle are now implemented; we cite the source and the deployed contracts, verify the aggregation's Byzantine-robustness against a machine-checked TLA+ spec, and state the coupled consensus-learning convergence result as a conjecture to be validated.

Keywords: paraconsistent logic, Belnap FOUR, federated learning, BlockDAG, meta-learning, LoRA composition, GhostDAG, distributed consensus, mixture of experts

0.2 1. Introduction

Federated learning and blockchain consensus solve structurally similar problems by different mechanisms. Both coordinate distributed agents toward shared objectives without central authority. Both must tolerate Byzantine participants. Both produce collective outputs (aggregated gradients; finalized blocks) from potentially contradictory local observations. Yet the two fields developed largely independently, with AI-incorporating blockchains treating consensus and computation as orthogonal.

This paper asks whether consensus and learning can be the same process. Specifically: can participating in BlockDAG consensus simultaneously contribute to a distributed learning objective, using the DAG's causal structure for gradient ordering, its blue-score mechanism for trust-weighted aggregation, and its finality checkpoints for learning synchronization?

We propose Paraconsistent Consensus, built on three contributions. First, an aggregation function based on Belnap's four-valued logic [14, 28] that handles contradictory model outputs without discarding minority contributions or collapsing into meaningless averages. Where classical federated averaging treats disagreement as noise, paraconsistent aggregation treats it as information: a signal that different nodes have learned different things, which a routing function can exploit. Second, a checkpoint-synchronized learning protocol that extends BFT finality checkpoints to include learning state, meta-model routing weights, per-node performance profiles, and LoRA adapter registries, turning the checkpoint into a learning synchronization point. Third, a bounded analysis of LoRA adapter composition, grounded in task arithmetic [17], interference mitigation [18], and the learn-less-forget-less tradeoff [19].

Implementation status. In February 2026 the extensions here were designed but unbuilt. That is no longer true, and this revision is careful to say exactly what exists. Claims marked **[Implemented]** cite a source file or a deployed contract; **[Specified]** are designed but unbuilt; **[Hypothesis]** are conjectures with a stated validation methodology. The consensus infrastructure itself, GhostDAG, BFT finality, LVM, precompiles, is described in Paper I.

0.3 2. Background and Related Work

0.3.1 2.1 BlockDAG Consensus

[Implemented] The Citrate consensus layer implements GhostDAG [1, 2] with BFT finality (Paper I; `core/consensus/src/ghostdag.rs:54`). Blocks reference up to 10 parents (one selected plus up to nine merge; `types.rs:179`), are classified into blue and red sets via the k-cluster rule (k=18; `types.rs:178`), and reach finality through committee signatures at checkpoints. The chain-40204 testnet block time is **approximately 2 seconds** (`testnet-config.toml:4`), and BFT checkpoints occur every **50 blocks** by default (`core/consensus/src/checkpoint.rs:101`). The February 2026 draft quoted 0.5-second blocks and 10-block checkpoints; both are corrected here against the running configuration. These parameters are the infrastructure on which this paper's extensions ride.

0.3.2 2.2 Federated Learning

FedAvg [6] established the canonical approach: N clients train locally, compute updates, and submit them to a server that aggregates via weighted averaging, preserving data privacy but introducing a central point of failure. FedProx [31] adds proximal regularization for heterogeneity; SCAFFOLD [32] corrects client drift via control variates; Byzantine-robust rules (Krum [33], trimmed mean,

coordinate-wise median) tolerate malicious updates. None eliminate the trusted aggregator. Recent surveys map the 2024–2025 aggregation landscape and its robustness tradeoffs [36, 37]. The aggregation in all these approaches collapses N updates into one, discarding disagreement structure: if two clients produce opposite gradients for the same layer, averaging cancels the signal. That is acceptable when disagreement is noise but wasteful when it reflects legitimate specialization, the core insight motivating paraconsistent aggregation.

0.3.3 2.3 LoRA and Adapter Composition

Low-Rank Adaptation (LoRA) [8] parameterizes updates as low-rank perturbations W + BA. Two properties matter here: base weights W remain frozen (enabling rollback by subtracting the adapter), and the update magnitude is bounded by the adapter's spectral norm. The composition literature reveals real challenges. Ilharco et al. [17] introduced task arithmetic and showed naive addition of task vectors causes interference when tasks modify overlapping parameters. Yadav et al. [18] proposed TIES-Merging to mitigate it via magnitude pruning and sign-conflict resolution. Biderman et al. [19] demonstrated that LoRA "learns less and forgets less" than full finetuning, better preserving base capabilities but with reduced task-specific performance. OPLoRA [34] finds interference concentrates in the dominant singular directions of pre-trained weights and proposes orthogonal projection. For Paraconsistent Consensus these findings mean LoRA is a structurally favorable starting point, frozen weights bound maximum regression, but multi-adapter composition over time requires active interference mitigation, addressed honestly in Section 5.

0.3.4 2.4 Paraconsistent Logic

Classical logic enforces explosion: from a contradiction, anything follows. In a network where honest nodes legitimately differ, due to model heterogeneity, different training data, or stochastic inference, this property is destructive. Paraconsistent logics tolerate contradiction without trivialization [14]. We adopt Belnap's four-valued logic [28, 28b], designed precisely for reasoning with contradictory information from multiple sources. Belnap's FOUR is a bilattice with four values: **T** (true, supported), **F** (false, contradicted), **B** (both, supported and contradicted), and **N** (neither, no evidence). It carries two orderings: a truth ordering (F N T and F B T) and an information ordering (N T B and N F B). Ginsberg's bilattice framework [38] generalizes this structure and is the right formal home for multi-source aggregation. Belnap's key insight: the information ordering tracks how much we know, independent of what we know. A state of B contains more information than T or F alone, because we know that sources disagree, which is itself informative. This maps directly to federated learning: when nodes agree, the aggregated state is T; when they disagree, it is B, and rather than discarding it we preserve it for the routing function, which can learn to route contradictory inputs to the most reliable expert.

0.3.5 2.5 Mixture of Experts

The Mixture-of-Experts (MoE) architecture [7] is the template for our routing: a gating function routes inputs to specialized sub-networks. In standard MoE both experts and gate are trained jointly. In our distributed setting the experts (node models) are trained independently and the gate (routing model) is trained at consensus checkpoints on aggregated embeddings. The attention mechanism [9] provides the input-dependent routing machinery, each node's embedding a key/value, the inference query the query.

0.4 3. The Paraconsistent Aggregation Function

0.4.1 3.1 Formal Framework

[Implemented] Let N be the set of nodes, each producing a d-dimensional embedding e for a reference input. Let b [0,1] be node i's normalized blue score (from GhostDAG's blue-set classification) and c [0,1] its per-dimension confidence (from softmax entropy). A classification function maps each node's embedding to a Belnap state per dimension j:

- **T (relevant):** c > _high and e is directionally consistent with the blue-score-weighted majority. Confident and agreeing.
- **F (irrelevant):** c > _high and e is directionally inconsistent with the majority. Confident but disagreeing, a signal of potential specialization, not error.
- **B (contradictory):** multiple nodes with comparable blue scores produce inconsistent embeddings with comparable confidence. Conflicting evidence, preserved rather than averaged.
- **N (unknown):** c < _low. The node is uncertain; this dimension contributes minimally.

This is no longer only a specification. The four Belnap states are implemented as `enum BelnapState { Neither = 0, True = 1, False = 2, Both = 3 }` in the on-chain aggregation precompile at address `0x0110` (`core/execution/src/precompiles/q16/belnap.rs:77-80`), whose header cites this paper's §3.2. The precompile bounds inputs at `MAX_N = 1024` nodes and `MAX_DIM = 1024` dimensions.

0.4.2 3.2 Aggregation Rule

[Implemented] The paraconsistent aggregation produces two outputs: an aggregated embedding e_agg and a state vector s {T, F, B, N} . The embedding is a trust-confidence-weighted combination

e_agg[j] = Σ (w · e [j]) / Σ w , w = c [j] · softmax(b /),

where controls trust concentration. Crucially, s is computed independently of e_agg: even when e_agg is a reasonable weighted average, s[j] = B records that this dimension carried contradictory evidence, information pure averaging would destroy. The routing model receives both e_agg and s. The in-process wrapper that drives this is `BelnapAggregator` (`core/learning-daemon/src/aggregator.rs`) which calls the precompile's `aggregate` entry (`aggregator.rs:234`); a learning-side module mirrors it at `core/learning/src/belnap.rs`. All arithmetic runs on the deterministic Q16.16 fixed-point path so the aggregation is bit-reproducible across nodes.

0.4.3 3.3 Comparison to Standard Aggregation

Table 1. Aggregation under node disagreement.

Method	Handles disagreement by	Information preserved	Limitation
FedAvg [6]	Weighted average	None (cancellation)	Opposing gradients cancel
Krum [33]	Selecting most representative	One viewpoint only	Discards minority views
Trimmed mean	Removing extremes, averaging rest	Central tendency	Removes potential experts

Method	Handles disagreement by	Information preserved	Limitation
Paraconsistent (ours)	Classifying into Belnap FOUR, preserving state vector	Full disagreement structure	Requires routing model to exploit

The tradeoff is explicit: paraconsistent aggregation preserves more information but requires a routing model capable of exploiting it. If the routing model is poorly trained, the extra information is wasted and the system reduces to weighted averaging. The value of the approach is therefore contingent on routing quality, a dependency we treat as a primary validation target.

Verified robustness property. The bucketed, coordinate-wise trimmed-mean reduction underlying robust aggregation is machine-checked. A TLA+ specification proves the reduction is deterministic and bit-reproducible per coordinate (`GradientAggregation.tla`, TLC-green), and an adversarial spec proves `ByzantineCannotFlipUnderHonestDominance`, that under honest-majority dominance Byzantine contributions cannot flip the aggregate, together with a window-closure liveness property (`GradientAggregationAdversarial.tla`, TLC-green as of 2026-06-27). These are properties of the aggregation kernel, not of the full coupled learning loop, whose convergence remains a conjecture (§7.2).

0.5 4. Checkpoint-Synchronized Learning

0.5.1 4.1 Extended Checkpoint Structure

[Implemented] We extend the BFT finality checkpoint (Paper I §2.3) with three committed fields: a routing-weights Merkle root, an adapter-registry Merkle root, and a performance-profile Merkle root. This is realized in `struct LearningCheckpoint` (`core/learning/src/checkpoint.rs:14`), whose fields are the routing-weights root (`:35`), the adapter-registry root (`:40`), the per-node metrics root (`:45`), and a macro-phase field (`:59`) whose comment cites this paper's §5; the type's header states it is "triggered by BFT finality." The extension is backward-compatible: non-learning nodes leave the fields empty and consensus treats all nodes identically for ordering. The default checkpoint interval of 50 blocks (~100 s at 2-second blocks) sets the learning synchronization cadence.

Architectural honesty note. The paraconsistent four-valued logic does **not** live inside `core/consensus/src/finality.rs`; that module is purely depth-based BFT finality. The Belnap aggregation is an execution-layer precompile plus a learning daemon, *invoked at* checkpoints, not a modification of the finality rule itself. Any reading of this paper that imagines finality is four-valued is mistaken; consensus is classical, and the four-valued reasoning sits one layer above it, consuming its blue scores and finality commitments.

0.5.2 4.2 The Routing Model

[Specified] The routing model is lightweight, not a full transformer: a small MLP or single-layer attention with roughly 100K–500K parameters. Inputs are the query embedding (d dimensions), e_agg (d), the Belnap state vector s (d, encoded 2-bit per dimension), and compressed per-node capability profiles (k « d per node). It outputs routing weights over participating nodes. A model this size retrains in milliseconds, making checkpoint-interval updates feasible. We reserve "second-order transformer", a full attention mechanism over node embeddings, treating each node's embedding as a token, for a long-term research goal; the initial routing model is deliberately simple so it can update reliably at consensus timescales.

0.5.3 4.3 Training Protocol

[**Implemented mechanism; Specified schedule**] The protocol runs in three phases. Phase 1 (Embedding Collection): nodes include embeddings in blocks (~3 KB for d=768, float32); the routing model learns which nodes are reliable and which specialize. Phase 2 (Routing Activation): once routing confidence passes a threshold, queries use learned routing rather than uniform distribution. Phase 3 (Adapter Generation): the routing model identifies node weaknesses and generates targeted LoRA adapters. The mechanism exists, LoRA adapters are created, applied, and verified (`core/learning/src/adapters.rs`, exercised end-to-end in `core/learning/tests/e2e_ooda_pipeline.rs`), registered on-chain via **`LoRAFactory.sol`** (deployed `0x6e564d22…`; note this is a contract, not the "0x1003 precompile" the February draft named), with mentor-mentee pairing at each checkpoint in `core/learning/src/mentor.rs:3` and on-chain matching in `MentorMatcher.sol`. The phase-transition criteria, calibration period, and adapter-generation frequency remain design parameters to be tuned on a live testnet (§8).

0.6 5. LoRA Adapter Composition: An Honest Assessment

0.6.1 5.1 Structural Advantages

LoRA provides three favorable properties. **Bounded perturbation:** each adapter perturbs by $\Delta W = BA$ with $\|BA\| \quad \|B\| \, \|A\|$, an explicit auditable bound. **Clean rollback:** frozen base weights mean an adapter can be removed by subtraction, restoring the exact prior state. **Reduced forgetting:** Biderman et al. [19] show LoRA mitigates catastrophic forgetting more effectively than full finetuning, weight decay, and dropout, though it does not eliminate it. The verify-and-rollback path is implemented (`adapters.rs`: `apply_lora`, `remove_lora`, `verify_lora_hash`), giving the safety mechanism a concrete home.

0.6.2 5.2 Composition Challenges

We do not claim composition avoids regression "by construction." The literature is clear that naive additive composition of adapters, or more generally task vectors, causes interference when they modify overlapping parameter subspaces [17, 18, 34]. **Task-arithmetic interference** [17]: adding task vectors (the difference between fine-tuned and pre-trained weights) enables flexible model editing, but the sum of multiple task vectors interferes in proportion to the cosine similarity between them; independently trained adapters can point in conflicting directions, particularly in overlapping regions. **LoRA-specific merging** [35]: work at ICLR 2025 shows standard merging methods (task arithmetic, TIES-Merging) transfer poorly to LoRA models compared to fully fine-tuned models, because LoRA's low-rank constraint concentrates updates in a lower-dimensional subspace where conflicts are more likely; KnOTS [35] addresses this through SVD-based alignment into a shared space before merging. **Dominant-subspace interference** [34]: OPLoRA identifies that forgetting concentrates in the dominant singular directions of pre-trained weights, the principal components encoding the most important learned representations, so an adapter that inadvertently modifies these directions causes disproportionate degradation. These three failure modes are why the implemented lifecycle keeps adapters individually reversible and the registry bounded, rather than assuming composition is safe.

0.6.3 5.3 Bounded Regression Guarantee

[**Hypothesis**] *Theorem 1 (Bounded Regression Under Perfect Routing).* Let M have weights W and let a ,…,a be adapters with perturbations ΔW ,…,ΔW . If routing achieves perfect specialization,

each query routed to exactly one adapter, no adapter applied outside its training distribution, then the maximum regression on any task t served by a is bounded by $\|\Delta W\| \cdot L$, with L the Lipschitz constant of the forward pass. Under imperfect routing with bounded error , the bound becomes $O(\cdot \max \|\Delta W\| \cdot L)$.

The guarantee degrades with routing error and creates a circular dependency: routing quality depends on the learning signal, which depends on adapter quality, which depends on routing. We acknowledge the circularity and propose mitigations, all consistent with the implemented lifecycle: (a) orthogonal adapter training [34]; (b) task-gated single-adapter selection per query; (c) periodic consolidation via SVD alignment [35]; (d) adapter retirement, supported by the `LoRAFactory` registry, bounding the number of concurrent adapters.

0.7 6. Block Structure and Integration

[Specified] The GhostDAG block is extended with three optional fields: an embedding e , computed by passing a shared reference input through the node's local model (~3 KB for d=768, float32); a confidence vector c [0,1] derived from softmax entropy (~3 KB); and a gradient commitment, a cryptographic hash of the node's gradient update revealed in the subsequent block to prevent front-running. These fields are optional and backward-compatible: non-participating nodes leave them empty, and the consensus protocol orders learning and non-learning blocks identically.

The per-block learning overhead is ~6–7 KB (embedding plus confidence, d=768), i.e. ~12–14 KB/s of additional bandwidth per node at 2-second blocks, well within the block-size limit. For larger embedding dimensions the overhead scales linearly, ~8–9 KB at d=1024 and ~16–18 KB at d=2048 per block. If overhead becomes constraining at higher block rates, embedding compression, float16 quantization, PCA dimensionality reduction, or learned compression, can reduce it 2–4×, at some cost to the fidelity of the Belnap state classification. The tradeoff between embedding fidelity and overhead is an engineering parameter to be tuned on the testnet, and it is one reason the block time (§2.1) is deliberately conservative: the 2-second cadence leaves headroom for the learning fields that a faster pure-transaction chain would not.

0.8 7. Safety, Liveness, and Convergence

0.8.1 7.1 Consensus Safety and Liveness

[Implemented, inherited from Paper I] The learning extensions do not modify consensus safety or liveness. Block ordering, blue/red classification, and BFT finality operate identically whether or not the learning fields are populated. Safety holds under $f < n/3$, liveness under $f < n/2$ (Paper I §2.3). The learning protocol rides on top of consensus, using its outputs as inputs to aggregation, and does not participate in the consensus mechanism itself, consistent with the architectural note in §4.1.

0.8.2 7.2 Learning Convergence

[Hypothesis] *Conjecture 1.* Under standard federated conditions, bounded gradient variance ^{2}, L-smooth loss, and convexity or the Polyak-Łojasiewicz condition, paraconsistent aggregation converges to a stationary point of the global loss. The argument: aggregation weights are normalized and bounded; blue-score weighting down-weights Byzantine contributions under honest majority; confidence weighting reduces variance. But this is not a proof. Three complications block a clean guarantee: (a) the blue score is a function of network dynamics, introducing a time-varying coefficient standard proofs do not handle; (b) the Belnap thresholding introduces a discontinuity

that may interfere with gradient-based optimization; (c) routing and node models update simultaneously, a coupled-dynamics problem. Proving convergence for the coupled consensus-learning system under Byzantine conditions is the primary open theoretical problem of this framework. The adversarial robustness of the *aggregation kernel* is separately machine-checked (§3.3), which bounds one term of the problem without closing it.

0.8.3 7.3 Slashing Extensions

[**Implemented**] The standard slashing conditions are extended with a learning-specific one: embedding manipulation, submitting embeddings detectably inconsistent with the node's registered model, detected by validator spot-checks that recompute the embedding for the same reference input. Slashing is enforced by `NematocystSlashing.sol` (deployed `0xfeb23abd…`, Paper IX), and detection uses the verifiable-inference infrastructure of Paper I §3.3 at the optimistic tier.

0.9 8. Experimental Methodology: What We Plan to Test

The aggregation primitive is implemented and its kernel is formally verified, but the network-scale learning hypotheses below have not been run. We state the methodology before collecting results, to avoid post-hoc rationalization.

H1 — Paraconsistent aggregation outperforms averaging. *Claim to test:* on a network of heterogeneous models, differing in architecture, training-data distribution, and capability profile, paraconsistent aggregation with Belnap state vectors produces higher routing accuracy than blue-score-weighted averaging alone. *Methodology:* deploy N = 50–100 nodes on a testnet, each hosting a different fine-tuned model (architectures spanning 1B–7B parameters), and use a held-out evaluation set with known domain labels. Compare routing accuracy, the fraction of queries routed to the best-performing node for that domain, across three conditions: uniform routing, blue-score-weighted averaging, and paraconsistent aggregation with Belnap state vectors. Report mean accuracy, per-domain accuracy, and calibration curves. *Baseline and negative result:* if paraconsistent aggregation does not beat weighted averaging by a statistically significant margin ($p < 0.05$, corrected for multiple comparisons), the added complexity of Belnap classification is not justified and the framework should simplify to weighted averaging, a valuable negative result we would report as readily as a positive one.

H2 — Adapter composition improves over time. *Claim to test:* over successive checkpoint intervals, adapters generated by the routing model improve node performance on weak domains without degrading performance on strong domains beyond the Theorem 1 bound. *Methodology:* run the full learning loop for an extended period (target ~10,000 checkpoints) and track per-node, per-domain accuracy at each checkpoint. Measure the weak-domain improvement rate, the strong-domain regression rate, the total adapter count and interference (cosine similarity between adapter weight vectors), and routing-model entropy as a measure of specialization, comparing runs with and without adapter consolidation. *Success criteria:* weak-domain accuracy improves monotonically within noise, and strong-domain regression stays below the Theorem 1 bound; if regression exceeds the bound, identify the routing error rate and determine whether the §5.3 mitigations are necessary.

H3 — Convergence under Byzantine conditions. *Claim to test:* the learning protocol converges to useful routing in the presence of up to $f < n/3$ Byzantine nodes submitting adversarial embeddings. *Methodology:* introduce Byzantine nodes at 10%, 20%, and 30% of the network with three adversarial strategies, random embeddings, strategically misleading embeddings designed to corrupt routing for specific input classes, and gradient poisoning that maximizes divergence from the true gradient, and measure convergence time, routing accuracy at convergence, and the

effectiveness of blue-score down-weighting at isolating Byzantine contributions. This is the hardest test. A strategic adversary who understands the aggregation function can craft embeddings that pass confidence checks while corrupting the routing model; we aim to characterize the attack surface and identify which strategies blue-score weighting mitigates and which require additional defense, not to claim robustness to all of them. The TLA+ adversarial result (§3.3) covers the aggregation kernel under honest-majority dominance; H3 tests the full coupled loop the kernel proof does not reach.

0.10 9. Biological Inspiration

The recursive learning architecture draws inspiration from cnidarian (jellyfish) neurobiology (Paper IX). The nerve net of *Aurelia aurita* achieves global coordination through purely local interactions, without central arbitration [20, 21]. Three design choices echo it: DAG topology (rather than a central aggregator) for embedding propagation, mirroring leaderless coordination; the checkpoint as an analogue of the through-conducting synchronization pulse; and the emphasis on preserving disagreement rather than forcing consensus, mirroring the nerve net's tolerance for simultaneous excitatory and inhibitory signals. These are design inspirations, not formal justifications; the system's properties are established by the distributed-systems and ML arguments of Sections 3–7.

0.11 10. Conclusion

Paraconsistent Consensus unifies blockchain consensus and federated learning: DAG topology for gradient ordering, blue scores for trust-weighted aggregation, finality checkpoints for learning synchronization, and Belnap's four-valued logic for information-preserving aggregation. The central claim, that preserving contradictory information via Belnap state vectors enables better routing than collapsing it by averaging, is now testable against an implemented primitive rather than a specification: the aggregation precompile exists (`0x0110`), its kernel's Byzantine robustness is machine-checked, the learning checkpoint commits routing and adapter state, and the adapter lifecycle runs against deployed contracts. What remains is the coupled-system convergence proof and the network-scale experiments of Section 8. We have kept the boundary between what is implemented, what is specified, and what is conjectured explicit, because in a framework that argues for preserving disagreement, it would be incoherent to collapse our own uncertainty.

0.12 Acknowledgments

Drafting and literature triage were assisted by AI systems; all mechanical claims were verified by the authors against the referenced source files, TLA+ specifications, and deployed contracts on chain 40204. This work received no external funding.

0.13 References

[1] Sompolinsky, Y., & Zohar, A. (2015). Secure high-rate transaction processing in Bitcoin. *Financial Cryptography and Data Security*, 507–527. [2] Sompolinsky, Y., & Zohar, A. (2018). PHANTOM and GHOSTDAG. *IACR Cryptology ePrint Archive.* [6] McMahan, B., et al. (2017). Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 1273–1282. [7] Shazeer, N., et al. (2017). Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. *ICLR.* [8] Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022.* [9] Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS 30.* [11] Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. *OSDI*, 173–186. [14]

Priest, G. (2006). *In Contradiction: A Study of the Transconsistent.* Oxford University Press. [17] Ilharco, G., et al. (2023). Editing models with task arithmetic. *ICLR.* [18] Yadav, P., et al. (2023). TIES-Merging: resolving interference when merging models. *NeurIPS.* [19] Biderman, D., et al. (2024). LoRA learns less and forgets less. *Transactions on Machine Learning Research.* (First author Dan Biderman.) [20] Anderson, P. A. V. (1989). *Evolution of the First Nervous Systems.* NATO ASI Series, Springer. [21] Weissbourd, B., et al. (2021). A genetically tractable jellyfish model for systems and evolutionary neuroscience. *Cell*, 184(24), 5854–5868. [22] Klosowski, L. (2025). The Medusa Paradigm. Gradient Papers No. IX (this series); earlier circulated as an unpublished internal note, Citrate Inc. [27] Pallasdies, F., et al. (2019). From single neurons to behavior in the jellyfish *Aurelia aurita. eLife*, 8, e50084. [28] Belnap, N. D. (1977). A useful four-valued logic. In Dunn, J. M., Epstein, G. (eds), *Modern Uses of Multiple-Valued Logic*, Episteme vol. 2, Reidel, Dordrecht. [28b] Belnap, N. D. (1977). How a computer should think. In Ryle, G. (ed), *Contemporary Aspects of Philosophy*, Oriel Press. [29] Dunn, J. M. (2019). Two, three, four, infinity: the path to the four-valued logic and beyond. In *New Essays on Belnap-Dunn Logic*, Springer. [30] Kaspa Network. (2025). KIP-14: Crescendo Hardfork. Activated May 5, 2025. [31] Li, T., et al. (2020). Federated optimization in heterogeneous networks. *MLSys.* [32] Karimireddy, S. P., et al. (2020). SCAFFOLD: stochastic controlled averaging for federated learning. *ICML.* [33] Blanchard, P., et al. (2017). Machine learning with adversaries: Byzantine-tolerant gradient descent. *NeurIPS.* [34] OPLoRA. (2025). Orthogonal Projection LoRA prevents catastrophic forgetting. arXiv:2510.13003. [35] KnOTS. (2025). Knowledge transfer via SVD for parameter-efficient multi-task model merging. *ICLR 2025.* [36] (Survey.) (2025). Federated Learning: a multi-level taxonomy of aggregation techniques. arXiv:2511.22616. *(Verify author list before final submission.)* [37] (Survey.) (2024). Advances in robust federated learning: heterogeneity considerations. arXiv:2405.09839. *(Verify author list before final submission.)* [38] Ginsberg, M. L. (1988). Multivalued logics: a uniform approach to reasoning in AI. *Computational Intelligence*, 4(3), 265–316. [39] (2025). Declarative distributed algorithms as axiomatic theories in three-valued modal logic over semitopologies. arXiv:2512.21137. *(Nearest work linking many-valued logic to distributed algorithms; verify at submission.)*

0.14 Appendix A: Protocol Parameters (reconciled against code, Aug 2026)

Parameter	Value	Source
Block time	~2 s (testnet)	Paper I; `testnet-config.toml:4`
k parameter	18	`core/consensus/src/types.rs:178`
Max parents	10 (1 selected + 9 merge)	`types.rs:179`
BFT committee / quorum	100 / 67	`checkpoint.rs:88,103`
Checkpoint interval	50 blocks	`checkpoint.rs:101`
Belnap aggregation precompile	`0x0110`	`precompiles/q16/belnap.rs:77`
Belnap input bounds	MAX_N 1024, MAX_DIM 1024	`belnap.rs:161,164`
LearningCheckpoint	routing/adapter/metrics roots	`learning/src/checkpoint.rs:14`
Adapter registry (on-chain)	`LoRAFactory.sol` `0x6e564d22…`	40204 addresses

Parameter	Value	Source
Slashing (embedding manipulation)	learning-specific	`NematocystSlashing.sol`

0.15 Appendix B: Learning Protocol Parameters (Specified, to be calibrated)

Parameter	Value	Rationale
Embedding dimension (d)	768 (default), configurable	matches common transformer hidden sizes
Embedding precision	float32 default, float16 optional	fidelity vs. bandwidth
Per-block embedding overhead	~3 KB (d=768, float32)	within block-size limit
Routing model size	100K–500K params	must retrain at checkpoint interval
Belnap threshold _high	0.8 (proposed)	calibrate on testnet
Belnap threshold _low	0.3 (proposed)	calibrate on testnet
Temperature	1.0 (default)	governs blue-score trust concentration
Adapter rank r	16 (default)	standard LoRA
Adapter consolidation interval	every 1,000 checkpoints	freshness vs. interference

---

This paper is part of the Gradient Papers series, published by Citrate Inc. Correspondence: Larry@citrate.ai